ThreatSense AI: Project Progress Report and Research Integration Plan

Authors: Moksh Lalwani (Roll No. 23BCP096) and Raj Joshi (Roll No. 23BCP508D), B.Tech Computer Science Engineering, Pandit Deendayal Energy University **Project:** ThreatSense AI, a multi-channel cybersecurity threat detection platform (minor project)
Report date: 21 October 2026 **Submitted to:** [Project guide]

1. Summary

ThreatSense AI analyses seven kinds of digital artifacts (URLs, emails, SMS, webpages, images, QR codes and files) and returns one explainable risk report. The engineering work is largely complete: 22 detectors across 5 threat domains, 5 intelligence engines, a Noisy-OR risk aggregator, an explainer, a FastAPI backend and a React dashboard, backed by 396 automated tests. Stages 0 to 11 of the plan are finished. Stage 12 (formal evaluation) is pending and Stage 13 (DeBERTa fine-tuning) is deferred.

Recent research (2024 to 2026) has moved in three directions: LLM agents that choose and combine tools [1][2], multi-agent debate and specialisation [3][4][10], and the discovery that LLM-based detectors can themselves be manipulated by prompt injection hidden in the content they inspect [6][7]. This report (A) documents the current work, (B) reviews the relevant papers, and (C) proposes a lean plan to integrate those ideas without rewriting the system: an evaluation harness, an aggregator ablation, a bounded LLM arbiter, and an injection-robustness suite.

The central claim we intend to test is that a rules-first, multi-channel pipeline with a bounded LLM arbiter resists prompt injection and LLM-rewritten phishing better than a plain single-prompt LLM classifier, at lower cost. This is a hypothesis, not a result.

2. Problem Statement

Modern attacks are multi-channel. The same scam can arrive as an email, an SMS, a QR code on a poster, or a screenshot. Most existing tools and most of the papers reviewed below analyse one channel at a time, such as a URL scanner or an email filter. ThreatSense AI unifies these attack surfaces into one pipeline with shared threat intelligence and a single risk score that explains why an item is dangerous.

3. Part A: Work Completed

3.1 System architecture

```
User input (7 artifact types)
        |
        v
Input Normalizer --> SafeFetcher (SSRF-safe HTTP)
        |
        v
AnalysisContext (shared state for all engines)
        |
   5 engines: NLP | URL | Web | Email | Media | Intel
        |
22 detectors consume engine signals and produce verdicts
        |
        v
Risk Aggregator (Noisy-OR per domain, weighted final score)
        |
        v
Explainer (defanged URLs, verdict summaries, redirect chains)
        |
        v
RiskReport (JSON) --> SQLite/Postgres --> React dashboard
```

3.2 Detection coverage

Domain	Detectors	Engines used
D1 Phishing and social engineering	D01 Email phishing, D02 Social engineering, D03 Spear phishing, D04 Scam and fraud, D05 Smishing, D06 Government scams, D07 Financial investment, D08 Recruitment scams, D09 Tech support scams	NLP
D2 Malicious URLs	D10 Typosquatting, D11 Brand impersonation, D12 Homoglyph attack, D13 Redirect chains, D14 Suspicious TLD	URL
D3 Malicious webpages	D15 Credential harvesting, D16 Fake auth pages, D17 Malicious webpages	Web, NLP
D4 Email threats	D18 Business email compromise, D19 Spam, D20 Suspicious attachments	Email, NLP

Domain	Detectors	Engines used
D5 Media threats	D21 QR phishing (quishing), D22 Screenshot scam	Media, NLP

3.3 Intelligence engines

- **NLP engine:** four tiers. A rules backend with a 15-signal lexicon and regex scorer (under 1 ms, no dependencies, always on); a zero-shot backend (BART-large-MNLI); a weighted ensemble (60% rules, 40% zero-shot); and a DeBERTa fine-tuned backend that is a stub until Stage 13.
- **URL engine:** feature extraction (length, entropy, digit ratio, subdomain depth), Levenshtein brand matching, homoglyph normalisation, suspicious-TLD scoring, and redirect-chain following.
- **Web engine:** BeautifulSoup DOM parsing, password and card field detection, form actions leaking to external or HTTP hosts, external script counts, JavaScript obfuscation, brand claims from title and h1, and link density.
- **Email engine:** full RFC 822 parsing, SPF/DKIM/DMARC checks, From and Reply-To mismatch, display-name brand spoofing, URL extraction from MIME parts, and attachment double-extension and MIME-mismatch checks.
- **Media engine:** asynchronous QR decoding (pyzbar) and OCR (pytesseract) with graceful fallback.
- **Intel engine:** local blocklists (domains, IPs, SHA-256 hashes, URLs) and optional live URLhaus lookups through the SSRF-safe fetcher.

3.4 Risk aggregation

Per-domain scores combine detector scores with Noisy-OR: domain_score = 1 - product(1 - s_i). Domain scores are then weighted into the final score: D1 30%, D2 30%, D3 20%, D4 15%, D5 5%. Hard floors apply: an intel blocklist hit sets the score to at least 0.80, and any CRITICAL evidence sets it to at least 0.70.

3.5 Engineering, testing and deployment

- **Stack:** FastAPI, Python 3.11, async SQLAlchemy with Alembic migrations, HTTPX, tldextract, rapidfuzz, Pillow, React 18 with Vite, Docker.
- **Design decisions:** a decorator-based detector registry (new detectors need no core changes); one shared AnalysisContext per request; graceful degradation (any failed engine is logged and the pipeline continues); rules-first NLP; Noisy-OR aggregation instead of simple averaging.

- **Testing:** 396 tests across API routes, context, normalizers, router, SafeFetcher (SSRF protection), URL detectors, NLP backends, and the web/email/media/intel engines and aggregator, with zero lint errors.
- **Deployment:** backend on Render (Docker), frontend on Vercel, source on GitHub, with seven real-world threat examples included for testing.

3.6 Development stages

Stage	Deliverable	Status
0 to 1	Scaffold, FastAPI skeleton, detector registry, engine interfaces	Done
2	Seven input normalizers, router, SafeFetcher, upload hardening	Done
3	URL engine and detectors D10 to D14	Done
4	NLP engine with four backends	Done (DeBERTa is a stub)
5	Detectors D01 to D09	Done
6 to 8	Web, Email and Media engines with D15 to D22	Done
9	Intel engine	Done
10	Risk aggregator and explainer	Done
11	All engines wired into /analyze, React dashboard	Done
12	Evaluation dataset, security review, API documentation	Pending
13	DeBERTa fine-tuning	Deferred

4. Current Gaps

The gap analysis below shapes what the research should be used for.

1. **No measured detection performance.** The 396 tests check that the code behaves as designed. They do not show precision, recall or false-positive rate on labelled real-world data.
2. **Hand-set aggregation.** The domain weights are chosen by hand, and Noisy-OR assumes independent evidence while detectors D01 to D09 share the same NLP signals, which can overcount correlated evidence.
3. **NLP tiers.** Only the rules tier is always available. The fine-tuned tier is untrained.

4. **Fixed brand list.** Brand impersonation relies on a built-in list. The literature notes that brand reference lists are costly to maintain and lag behind new brands [2].
5. **No adversarial testing.** Text from HTML, email bodies and OCR of screenshots feeds the NLP engine. Any LLM tier that reads this text is exposed to injected instructions [6].
6. **English-centric lexicon.** Multilingual performance is untested, and the literature reports drops for LLM detectors in multilingual settings [7].

5. Part B: Literature Review

All results below are as reported by the original authors. We have not reproduced them.

5.1 LLM agents for phishing detection

PhishAgent (Cao et al., AAAI 2025 Oral) combines online and offline knowledge bases with multimodal LLMs for phishing webpage detection [1].

MemoPhishAgent (Chen et al., 2026) is a multi-modal LLM agent that selects and orders phishing-specific tools based on the evidence collected so far, instead of following a fixed workflow [2]. It has five tools (page crawl, screenshot check, image check, search, and extraction of child links) and an episodic memory that stores past reasoning traces. A three-tier policy governs memory use: full investigation when nothing similar exists, past traces as in-context examples on partial matches, and majority vote on strong matches. The authors report a recall gain of 13.6% over baselines on two public datasets, a 20% gain on a real-world social-media set (SocPhish, 2,765 URLs), up to 27% recall gain from memory, and production use on about 60,000 high-risk URLs per week at 91.44% recall [2]. One result matters for us: a plain URL-only transformer baseline scored higher accuracy than the agent on the TR-OP benchmark (0.9814 against 0.9303), which suggests that conventional benchmarks can be solved by URL patterns alone, and that harder in-the-wild data is needed to show value [2].

PhishDebate (Li et al., IEEE BigData 2025) argues that single-agent LLM classification is prone to hallucination and limited in interpretability, and proposes a modular multi-agent debate framework [3]. **Debate-Driven Multi-Agent LLMs** (Nguyen et al., ISDFS 2025) applies a similar debate idea to phishing email text [10].

MultiPhishGuard (Xue et al., ACM CCS 2025) uses agents that specialise in different parts of an email (text, URL, metadata) and adds an agent that rewrites emails to imitate current phishing tactics for adversarial training [4].

5.2 Reference-based and knowledge-based detection

KnowPhish (Li et al., USENIX Security 2024) enhances reference-based detection by combining LLMs with a multimodal knowledge graph of brands [5]. This is the research version of what our D11 brand check does with a hand-written list.

5.3 Robustness: attacks on LLM-based detectors

Clouding the Mirror (Koide, Nakano and Chiba, 2026) is presented as the first systematic evaluation of prompt injection against LLM-based phishing detection [6]. Its key idea is perceptual asymmetry: instructions hidden in HTML metadata, comments, invisible elements, tiny or near-background text, embedded images, or URL components are invisible to a human visitor but readable by the LLM. The authors define five attack techniques (legitimate pretexting, role hijacking, safety-policy triggering, tool or function hijacking, and content flooding) and six attack surfaces. On 2,000 synthetic phishing pages, the attack success rate in a basic prompt mode ranged from 39.9% (GPT-5) to 84.7% (Llama 4 Maverick). Legitimate pretexting, such as claiming to be a security training site, was the strongest technique, and invisible text surfaces worked best [6].

The proposed defence, **InjectDefuser**, combines three layers: prompt hardening (untrusted content wrapped in delimiters with unpredictable identifiers, a parsed URL structure, and instructions to ignore directives found in the content), allowlist-grounded retrieval of legitimate brand domains, and strict output-schema validation [6]. It reduced the GPT-5 attack success rate to 0.3% on the synthetic set and to 8.9% on prompt-injected real phishing pages, but it did not eliminate the weakness, and the authors report a higher false-positive rate on a weaker model (25.2% for Gemma 3). They also note they found no clear in-the-wild examples of such attacks yet, so the threat is demonstrated rather than observed [6].

LLM-PEA (Hassan et al., 2025) evaluates three frontier LLMs on phishing email and finds accuracy above 90%, but also vulnerability to prompt injection, adversarial rewriting and multilingual inputs [7]. MemoPhishAgent's own test shows the same pattern: under injection a plain multimodal LLM fell from 0.83 to 0.50 accuracy, while the agent fell from 0.96 to 0.86, which the authors attribute to evidence being aggregated across several tools and past cases [2].

5.4 QR phishing (quishing)

Trad and Chehab propose detecting quishing from the QR code's structure and pixel patterns without decoding the payload, noting that QR codes can carry data other than URLs, such as Wi-Fi credentials and payment information [8]. They built a dataset of 9,987 QR codes generated from 5,000 benign and 5,000 phishing URLs, and their best model (XGBoost) reached an AUC of about 0.91 [8]. QRiS extracts 24 structural features and

emphasises transparent, explainable classification [9]. Both datasets are synthetic QR codes generated from URL lists, so these results show feasibility rather than performance on photographed real-world codes.

5.5 What the literature leaves open

The papers above mostly study one channel (URL and webpage [1][2][5][6] or email [4][7][10]). We found no work that unifies URLs, email, SMS, QR codes and screenshots in one explainable pipeline with a deterministic first pass, which is where ThreatSense AI can contribute. Robustness evidence for such hybrid designs is also thin, since the injection studies focus on pure-LLM detectors [6][7].

6. Part C: Integrating the Research into ThreatSense AI

The principle is to add each research idea as a switchable module on top of the existing registry and context design. With a module disabled, the system behaves exactly as it does today.

6.1 How the research maps onto our system

Aspect	What the literature does	ThreatSense today	Planned addition
Channels	One channel per system [1][2][4][5]	Seven artifact types, one score	Keep; present as the main differentiator
Decision logic	LLM agent picks tools adaptively [1][2]	Fixed rules, Noisy-OR	Bounded LLM arbiter on borderline cases, with our detectors as its evidence
Debate and roles	Debate between agents [3][10], specialised agents [4]	Single pass	Optional analyst and skeptic roles inside the arbiter
Brand knowledge	Knowledge graph and reference lists [5][6]	Hard-coded brand list	Data file mapping each brand to official domains
Memory	Episodic memory [2]	Reports stored, not reused	Optional: retrieve similar past reports from the existing database
Injection robustness	Attacks and defences studied [6][7]	Untested	Injection test suite, hardened arbiter prompt, output validation

Aspect	What the literature does	ThreatSense today	Planned addition
QR analysis	Structural features without decoding [8][9]	Decode, then analyse URL	Optional structural pre-decode signal for D21
Evaluation	Public datasets, recall, F1, AUC [2]	Unit tests only	Evaluation harness with calibration and ablations

6.2 Target architecture

```
... existing pipeline (engines + 22 detectors) ...
        |
        v
Risk Aggregator (mode switch: noisy_or | mean | max | weighted_mean)
        |
        +-- score outside grey band --> final score (unchanged)
        |
        +-- score inside grey band ----> LLM Arbiter (off by default)
                                           - input: structured detector
evidence
                                           - untrusted text in delimited
blocks
                                           - output: validated JSON, bounded
adjustment
                                           - hard floors always preserved
        |
        v
Explainer (adds arbiter rationale) --> RiskReport

Offline: eval/ harness, ablations, injection suite
```

6.3 Implementation phases

Phase A: Evaluation harness (Stage 12). A script runs the same internal analysis function the API uses over a labelled CSV and caches per-detector scores. It reports precision, recall, F1, ROC-AUC, PR-AUC, recall at 1% and 5% false-positive rate, Brier score and expected calibration error, with bootstrap confidence intervals. This closes the largest gap in Section 4 and is the foundation for every other phase.

Phase B: Aggregator ablation. An AGGREGATION_MODE setting (noisy_or by default, plus mean, max and weighted_mean) with the hard floors kept in every mode, and leave-one-domain-out runs using the cached scores. This tests whether Noisy-OR and the

hand-set weights actually help, and whether correlated NLP detectors cause overcounting.

Phase C: Bounded LLM arbiter. A new engine, disabled by default, invoked only when the final score falls in a configurable grey band. It follows the research as described in 6.4.

Phase D: Robustness suite. Adapt the taxonomy from [6] to our inputs and compare the pipeline variants under attack (6.6), plus a test with LLM-rewritten phishing text [7].

Phase E (optional, if time permits). A brand-to-domain data file [5][6], episodic retrieval from the existing database [2], and a structural QR signal for D21 [8][9].

6.4 Arbiter design rules

1. **Structured evidence first.** The arbiter receives detector verdicts and engine signals, plus a short truncated excerpt of untrusted content. It never receives raw pages or emails as instructions.
2. **Explicit trust boundaries.** Untrusted text is enclosed in delimiters with unpredictable identifiers, the URL is passed in parsed form, and the prompt states that directives found inside the content are to be ignored [6].
3. **Bounded influence.** The arbiter can shift the score by only a small configured amount and can never push it below the existing hard floors (0.80 for an intel hit, 0.70 for CRITICAL evidence). A successful injection therefore cannot flip a known-bad verdict.
4. **Validated output.** The response must match a fixed JSON schema. Anything else is discarded and the original score stands [6].
5. **Graceful degradation.** If the arbiter is disabled, unavailable or times out, the pipeline continues as before, consistent with our existing design.
6. **Interpretability.** The arbiter's rationale is stored in the RiskReport. An analyst-and-skeptic variant can be tried, inspired by debate-based approaches [3][10].

6.5 Research questions and hypotheses

- **RQ1.** What are the real precision, recall and calibration of the rules-first pipeline on labelled data, and how do they change with the arbiter?
- **RQ2.** Does Noisy-OR outperform simple averaging, max, and a weighted mean?
- **RQ3.** Under prompt injection and LLM rewriting, does the bounded arbiter degrade less than a plain single-prompt LLM classifier?

The hypotheses are that the arbiter improves recall on borderline cases at comparable false-positive rate, and that the bounded design is more injection-resistant than a plain LLM classifier, motivated by [2] and [6]. Either hypothesis may be refuted, and an honest negative result will be reported as such.

6.6 Evaluation protocol

- **Data.** The agent paper [2] uses TR-OP (5,000 phishing URLs from OpenPhish and 5,000 benign URLs from the Tranco top list), DynaPD, and SocPhish. Their public availability must be checked. As a fallback we will assemble labelled URL sets from phishing feeds and the Tranco list, use public email corpora for email, and use the released QR dataset from [8]. Licences will be confirmed before use.
- **Baselines.** Rules-only, rules plus zero-shot ensemble, a lightweight URL-only classifier (because [2] shows URL patterns can be strong on standard benchmarks), a plain single-prompt LLM classifier as a control, and the full pipeline with the arbiter.
- **Metrics.** Precision, recall, F1, ROC-AUC, PR-AUC, recall at fixed false-positive rate, Brier score, ECE, latency, arbiter call rate and per-sample LLM cost.
- **Injection suite.** Techniques from [6] (legitimate pretexting, role hijacking, safety-policy triggering, tool or function hijacking, content flooding) applied to our surfaces: HTML metadata, comments, invisible and visible text, email bodies, URL components, and OCR text from images. Attack success is defined as in [6]: any known-malicious item not judged malicious, counting refusals and malformed outputs. We also test whether the defence raises false positives on benign long-tail sites, since [6] reports this risk for weaker models.

7. Improvement Summary and Timeline

Priority	Improvement	Research basis	Effort
1	Evaluation harness with calibration	[2][6] reporting practice	Small
2	Aggregator ablation	Own critique of Noisy-OR	Small
3	Bounded LLM arbiter	[1][2][3][6]	Medium
4	Injection and rewriting robustness suite	[6][7]	Small to medium
5	Brand data file	[5][6]	Small
6	Episodic memory, QR structural signal	[2][8][9]	Medium, optional

Proposed schedule (to be adjusted to the submission deadline): week 1, Phases A and B; week 2, Phase C; week 3, Phase D; week 4, results analysis and report. Phase E only if earlier phases finish early.

8. Risks and Limitations

- The arbiter may not improve results. The aim is a measured answer, not a guaranteed gain.
- LLM calls add latency and cost. One study measured about 14.65 seconds per website with its defence on GPT-5 [6], so the arbiter is restricted to the grey band and optional.
- Defences against injection reduce but do not remove the problem, and can raise false positives on weaker models [6].
- Results from the papers come from different datasets and setups, so they are context, not targets to match.
- Public QR datasets are synthetic [8], so quishing results will be indicative only.
- Dataset availability and licences are unconfirmed and may force a smaller, self-built evaluation set.

9. Future Scope

DeBERTa fine-tuning on a labelled phishing corpus (Stage 13), a multilingual and Hinglish evaluation set covering UPI, KYC and job scams, learned stacking of detector outputs, episodic memory, and structural QR analysis.

10. References

[1] T. Cao, C. Huang, Y. Li, H. Wang, A. He, N. Oo and B. Hooi, PhishAgent: A Robust Multimodal Agent for Phishing Webpage Detection, AAAI Conference on Artificial Intelligence, 2025 (Oral). arXiv:2408.10738.

[2] X. Chen, M. Kafai, H. Liu, P. Habas, T. Yuan and X. Zhang, MemoPhishAgent: Memory-Augmented Multi-Modal LLM Agent for Phishing URL Detection, arXiv:2602.21394, 2026.

[3] W. Li, S. Manickam, Y.-W. Chong and S. Karuppayah, PhishDebate: An LLM-Based Multi-Agent Framework for Phishing Website Detection, IEEE International Conference on Big Data, 2025. arXiv:2506.15656.

[4] Y. Xue, E. Spero, Y. S. Koh and G. Russello, MultiPhishGuard: An LLM-based Multi-Agent System for Phishing Email Detection, ACM CCS 2025, Taipei. arXiv:2505.23803.

[5] Y. Li, C. Huang, S. Deng, M. L. Lock, T. Cao, N. Oo, H. W. Lim and B. Hooi, KnowPhish: Large Language Models Meet Multimodal Knowledge Graphs for Enhancing Reference-Based Phishing Detection, USENIX Security Symposium, 2024. arXiv:2403.02253.

[6] T. Koide, H. Nakano and D. Chiba, Clouding the Mirror: Stealthy Prompt Injection Attacks Targeting LLM-based Phishing Detection, arXiv:2602.05484, 2026 (v2).

[7] N. Hassan, P. BusiReddyGari, H. Zhao, Y. Ren, J. Xu and S. Zhang, LLM-PEA: Leveraging Large Language Models Against Phishing Email Attacks (v2 title: Phishing Email Detection Using Large Language Models), arXiv:2512.10104, 2025.

[8] F. Trad and A. Chehab, Detecting Quishing Attacks with Machine Learning Techniques Through QR Code Analysis, arXiv:2505.03451, 2025.

[9] M. W. Akram et al., QRiS: A Preemptive Novel Method for Quishing Detection Through Structural Features of QR, arXiv:2510.17175, 2025.

[10] N. T. V. Nguyen, F. D. Childress and Y. Yin, Debate-Driven Multi-Agent LLMs for Phishing Email Detection, International Symposium on Digital Forensics and Security (ISDFS), 2025. arXiv:2503.22038.

All arXiv papers can be opened at arxiv.org/abs/ followed by the identifier.